Faculté polydisciplinaire d'Errachidia

Année Universitaire : 2025-2026

Programmation C 1

Licence d'Excellence Informatique Appliquée à la Gestion · Semestre 5

Pr. Mounir Hanine

bonjour.c

```c
#include <stdio.h>

int main(void)
{
    printf("Bonjour !\n");
    return 0;
}
```

Plan

1. Introduction au langage C
2. Concepts de base
3. Structures de contrôle
4. Tableaux
5. Chaînes de caractères
6. Pointeurs
7. Fonctions

1

Introduction au langage C

- Du problème au programme
- La compilation, étape par étape
- Votre premier programme
- Lire les messages d'erreur sans paniquer

Un programme, c'est une suite d'instructions précises

Chaque soir, le gérant d'une librairie additionne à la main les ventes, calcule la TVA et note le total. Vingt minutes, et parfois des erreurs.

Un programme peut le faire en une fraction de seconde, sans erreur. Mais l'ordinateur ne sait pas ce qu'est une vente : il faut tout lui expliquer, étape par étape.

À retenir

Un langage de programmation sert à décrire ces étapes dans une forme que l'ordinateur peut exécuter.

L'analogie du traducteur

Votre lettre en français = le code source, écrit en C

Le traducteur = le compilateur

La version traduite = le programme exécutable, que la machine comprend

De la source à l'exécutable : trois étapes

1

Préprocesseur

Traite les lignes #include et #define : insère les bibliothèques

2

Compilateur

Vérifie la grammaire et traduit en langage machine

3

Éditeur de liens

Assemble votre code et les fonctions prêtes (printf…)



Une fois compilé, l'exécutable se lance autant de fois qu'on veut, sans recompiler, tant que le code source ne change pas.

Un langage né en 1972, toujours partout

1972

Dennis Ritchie crée le C aux laboratoires Bell pour écrire le système Unix.

- **Systèmes d'exploitation :** Linux, une grande partie de Windows et de macOS
- **Logiciels embarqués :** voitures, distributeurs de billets, terminaux de paiement
- **Langages descendants :** C++, Java, C#, PHP et JavaScript reprennent sa syntaxe

À retenir

Le C est un langage compilé, rapide et proche de la machine. Sa grammaire se retrouve dans beaucoup d'autres langages.

Toujours réfléchir à l'algorithme avant de coder

Un algorithme est une méthode décrite en français clair, comme une recette : des données, des étapes dans l'ordre, un résultat.

1	2	3	4	5	6
Lire le prix HT	Lire la quantité	Montant HT = prix × quantité	TVA = montant HT × 0,20	TTC = montant HT + TVA	Afficher le TTC

Piège classique

Taper du code au hasard en espérant que cela marche. Écrivez d'abord les étapes sur papier.

Choisir son environnement de travail

Outil	Système	Remarque
Code::Blocks (avec MinGW)	Windows, Linux	Gratuit et simple : utilisé en TP
Dev-C++	Windows	Très léger, adapté aux débutants
VS Code + GCC	Windows, Linux, macOS	Plus moderne, un peu de configuration
Compilateurs en ligne	Navigateur	Pratique pour s'entraîner sur téléphone

Piège classique

Télécharger Code::Blocks sans le compilateur. Choisissez la version dont le nom contient « mingw ».

Votre premier programme, ligne par ligne

bonjour.c

```c
#include <stdio.h>

int main(void)
{
    printf("Bonjour, bienvenue en C !\n");
    return 0;
}
```

- **#include <stdio.h> :** la bibliothèque qui contient printf
- **int main(void) :** le point de départ de tout programme
- **{ } :** délimitent les instructions de la fonction
- **printf :** affiche le texte ; \n passe à la ligne
- **return 0; :** fin normale du programme

Piège classique

Oublier le point-virgule. Le compilateur signale souvent l'erreur à la ligne suivante : regardez d'abord la fin de la ligne précédente.

La silhouette de tous vos programmes

modele.c

```c
#include <stdio.h>      /* 1. bibliotheques */

int main(void)          /* 2. fonction principale */
{
    /* 3. declaration des variables */

    /* 4. lecture, calculs, affichage */

    return 0;           /* 5. fin du programme */
}
```

Les commentaires

/* … */ sur plusieurs lignes

// … jusqu'à la fin de la ligne

Un bon commentaire

explique pourquoi, pas ce que fait la ligne : « on passe au client suivant ».

Avant chaque exercice, recopiez cette structure, puis remplissez les parties 3 et 4.

Lire un message d'erreur sans paniquer

Message (GCC)	Ce qu'il veut dire	Que faire
`expected ';' before`	Il manque un point-virgule	Regarder la ligne précédente
`implicit declaration of 'printf'`	Fonction inconnue	Ajouter #include <stdio.h>
`'x' undeclared`	Variable non déclarée	La déclarer ou corriger le nom
`expected '}' at end of input`	Accolade non fermée	Compter les { et les }



À retenir

Corrigez toujours la première erreur, puis recompilez. Une seule faute peut provoquer dix messages qui disparaissent ensemble. Un avertissement (warning) ne bloque pas, mais annonce souvent un résultat faux.

Exercice : trouvez les quatre fautes

fautes.c

```c
#include <stdio>

int main(void)
{
    printf("Total de la journee\n")
    printf("Merci de votre visite\n);
    return 0;
```

Corrigé

1. #include <stdio.h> : il manque .h
2. Point-virgule après le premier printf
3. Guillemet fermant dans le second printf
4. Accolade } finale de main

Exercices 1.1 à 1.4 corrigés dans le polycopié : carte de visite, ticket de caisse, questions de cours.

2

Concepts de base du langage C

- Variables et types
- Constantes et affectation
- Opérateurs arithmétiques
- Lire avec scanf, afficher avec printf

Une variable est un casier étiqueté

Dans un entrepôt, chaque marchandise est rangée dans un casier qui porte une étiquette. Une variable est un emplacement de la mémoire, avec :

Un nom
choisi par vous

`quantite`

Un type
la sorte de valeur

`int`

Une valeur
le contenu du moment

`12`

```c
int quantite = 12;      /* declaration + initialisation */
```

Déclarer, initialiser et bien nommer

```c
int quantite;
float prixUnitaire = 7.50;
char categorie = 'A';
int stock = 100, entrees = 40;
```

> **Piège classique**
>
> Une variable non initialisée contient n'importe quoi. Initialisez toujours vos totaux et vos compteurs à 0.

Nom	Valide ?
`montantHT`	Oui
`taux_tva`	Oui
`2emeClient`	Non : commence par un chiffre
`prix unitaire`	Non : contient un espace
`quantité`	Non : contient un accent

Le C distingue majuscules et minuscules : Total et total sont deux variables différentes.

Les types de base et leur usage en gestion

Type	Contenu	Taille habituelle	Exemple de gestion
`int`	Entier	4 octets	Quantité en stock
`long`	Grand entier	4 ou 8 octets	Numéro de facture
`float`	Réel, ~7 chiffres	4 octets	Prix d'un article
`double`	Réel, ~15 chiffres	8 octets	Montant comptable
`char`	Un caractère	1 octet	Catégorie 'A', 'B', 'C'

À retenir

Pour les montants financiers, préférez double à float : les arrondis de float apparaissent vite sur les gros montants.

Constantes et affectation

```c
const double TAUX_TVA = 0.20;
#define NB_MOIS 12

int stock;
stock = 150;
stock = stock - 30;   /* stock vaut 120 */
```

Constante

Une valeur qui ne change jamais. En MAJUSCULES par convention : une seule ligne à modifier si elle change.

Affectation =

Se lit de droite à gauche : « calcule stock − 30, puis range le résultat dans stock ».

À retenir

Le signe = ne veut pas dire « est égal à », il veut dire « reçoit ».

Les opérateurs arithmétiques

Opérateur	Rôle	Exemple	Résultat
`+  -  *`	Addition, soustraction, multiplication	`15 * 4`	60
`/`	Division entre entiers	`15 / 4`	3
`/`	Division avec un réel	`15.0 / 4`	3.75
`%`	Reste de la division entière	`15 % 4`	3

7 / 2 = 3

et non 3,5 : les deux nombres sont entiers

cartons.c

```c
int articles = 250;
int cartons = articles / 12;   /* 20 */
int reste   = articles % 12;   /* 10 */
```

Piège classique

Pour 3,5, écrivez 7.0 / 2 ou (double)7 / 2.

Afficher avec printf

```
int quantite = 3;
double prix = 45.5;
printf("%d articles a %.2f DH\n", quantite, prix);
```

affichage

```
3 articles a 45.50 DH
```

Format	Affiche
%d	un int
%f	un float ou double
%.2f	2 décimales
%8.2f	largeur 8, à droite
%c	un caractère
%%	le symbole %

Chaque % est remplacé, dans l'ordre, par la valeur d'une variable placée après le texte.

Lire au clavier avec scanf

```c
int quantite;
double prix;

printf("Quantite : ");
scanf("%d", &quantite);
printf("Prix : ");
scanf("%lf", &prix);
```

Type	Format scanf
`int`	`%d`
`float`	`%f`
`double`	`%lf`
`char`	`" %c"`

Le & signifie « l'adresse de » : scanf doit savoir où ranger la valeur. Le chapitre 6 l'expliquera en détail.

Piège classique

Oublier le & (le programme plante) ou lire un double avec %f au lieu de %lf (valeur fausse).

Exemple complet : la facture

facture.c

```c
#define TAUX_TVA 0.20

int main(void)
{
    double prixHT, montantHT, tva, ttc;
    int quantite;
    printf("Prix HT : ");  scanf("%lf", &prixHT);
    printf("Quantite : "); scanf("%d", &quantite);
    montantHT = prixHT * quantite;
    tva = montantHT * TAUX_TVA;
    ttc = montantHT + tva;
    printf("TTC : %.2f DH\n", ttc);
    return 0;
}
```

execution

```
Prix HT : 125.50
Quantite : 4
TTC : 602.40 DH
```

Le calcul

502,00 × 0,20 = 100,40 DH de TVA

502,00 + 100,40 = 602,40 DH TTC

Exercice : convertir une durée

Un employé a travaillé 135 minutes sur un projet. Affichez cette durée en heures et minutes.

duree.c

```c
int totalMinutes, heures, minutes;

printf("Duree en minutes : ");
scanf("%d", &totalMinutes);

heures  = totalMinutes / 60;
minutes = totalMinutes % 60;

printf("%d h %d min\n", heures, minutes);
```

2 h 15 min

```
135 / 60 = 2
135 % 60 = 15
```

À retenir

/ et % sur des entiers : le duo idéal pour les conversions.

3

Structures de contrôle

- Conditions : if, else if, switch
- Boucles : for, while, do ... while
- break et continue
- Boucles imbriquées

Comparer et combiner des conditions

Opérateur	Sens
`==`	égal à
`!=`	différent de
`<` `<=`	inférieur (ou égal)
`>` `>=`	supérieur (ou égal)

Logique	Sens	Exemple
`&&`	ET	`age >= 18 && revenu > 4000`
`||`	OU	`fidele || montant > 5000`
`!`	NON	`!(stock > 0)`

Piège classique

= affecte, == compare. if (stock = 0) met stock à 0 et la condition est toujours fausse. Relisez toujours vos conditions.

if … else : l'alerte de réapprovisionnement

alerte.c

```c
int stock, seuil = 20;

printf("Stock actuel : ");
scanf("%d", &stock);

if (stock < seuil)
{
    printf("Alerte : commander %d unites\n",
           seuil * 3 - stock);
}
else
{
    printf("Stock suffisant.\n");
}
```

si … alors … sinon

Si la condition est vraie, le premier bloc s'exécute ; sinon, c'est le bloc else.

execution

```
Stock actuel : 12
Alerte : commander 48 unites
```

else if : une remise par tranches

Commande	Remise
Moins de 1 000 DH	0 %
1 000 à < 5 000 DH	5 %
5 000 à < 20 000 DH	10 %
20 000 DH et plus	15 %

remise.c

```
if (montant < 1000)
    taux = 0;
else if (montant < 5000)
    taux = 5;
else if (montant < 20000)
    taux = 10;
else
    taux = 15;
```

À retenir

Les conditions sont testées dans l'ordre : seul le premier bloc vrai s'exécute. Inutile de répéter montant >= 1000 dans le deuxième test.

switch : le choix dans un menu

menu.c

```c
switch (choix)
{
    case 1:
        printf("Saisie d'une vente\n");
        break;
    case 2:
        printf("Consultation du stock\n");
        break;
    default:
        printf("Choix invalide\n");
}
```

- **case :** une valeur possible de la variable
- **break :** sort du switch
- **default :** toutes les autres valeurs

Piège classique

Oublier un break : le programme continue dans le case suivant et affiche plusieurs messages.

Trois boucles, trois usages

`for`

Le nombre de tours est connu à l'avance

Exemple : 30 jours, 12 mois, 50 clients

`while`

On répète tant qu'une condition reste vraie

Exemple : Jusqu'à ce que le capital double

`do … while`

Comme while, mais au moins une fois

Exemple : Saisie contrôlée, menu

À retenir

Une boucle doit toujours avoir une condition de sortie qui finit par devenir fausse.

for : le total des ventes de la semaine

semaine.c

```c
int jour;
double vente, total = 0;

for (jour = 1; jour <= 7; jour++)
{
    printf("Ventes du jour %d : ", jour);
    scanf("%lf", &vente);
    total += vente;
}

printf("Total : %.2f DH\n", total);
printf("Moyenne : %.2f DH\n", total / 7);
```

Se lit ainsi

on part de jour = 1,

on continue tant que jour <= 7,

à chaque tour jour++

À retenir

total, initialisé à 0 et augmenté à chaque tour, est un accumulateur.

while : quand le capital aura-t-il doublé ?

capital.c

```c
double capital = 10000, objectif = 20000;
int annees = 0;

while (capital < objectif)
{
    capital = capital * 1.05;
    annees++;
}

printf("Double en %d ans\n", annees);
```

15 ans

pour 10 000 DH placés à 5 % par an :
20 789,28 DH

Piège classique

Si rien ne fait évoluer la condition, la boucle est infinie. Arrêtez avec Ctrl + C.

On ne sait pas combien de tours il faudra : c'est le cas typique de while.

do ... while : contrôler une saisie

saisie.c

```c
int quantite;

do
{
    printf("Quantite (1 a 500) : ");
    scanf("%d", &quantite);
} while (quantite < 1 || quantite > 500);

printf("Commande : %d unites\n", quantite);
```

```
Quantite (1 a 500) : 900
Quantite (1 a 500) : 120
Commande : 120 unites
```

D'abord on fait, ensuite on teste

On demande, puis on redemande tant que la réponse n'est pas valable.

Piège classique

Oublier le point-virgule après le while final.

break et continue

factures.c

```c
while (1)
{
    printf("Montant (0 pour finir) : ");
    scanf("%lf", &montant);
    if (montant == 0)
        break;          /* fin de la saisie */
    if (montant < 0)
        continue;       /* facture ignoree */
    total += montant;
    nb++;
}
```

break

sort de la boucle immédiatement

continue

abandonne ce tour, passe au suivant

while (1) tourne sans fin : on en sort par break. Construction légitime si la sortie est claire.

Exercice : tableau d'amortissement linéaire

Machine achetée 60 000 DH, amortie sur 5 ans. Afficher dotation, cumul et VNC pour chaque année.

amortissement.c

```c
double valeur = 60000, cumul = 0;
int duree = 5, annee;
double dotation = valeur / duree;

for (annee = 1; annee <= duree; annee++)
{
    cumul += dotation;
    printf("%5d %10.2f %10.2f %10.2f\n",
           annee, dotation, cumul, valeur - cumul);
}
```

Année	Dotation	Cumul	VNC
1	12 000	12 000	48 000
2	12 000	24 000	36 000
3	12 000	36 000	24 000
4	12 000	48 000	12 000
5	12 000	60 000	0

Montants en DH.

4

Tableaux

- Ranger plusieurs valeurs sous un seul nom
- Parcourir avec une boucle
- Somme, maximum, recherche, tri
- Tableaux à deux dimensions

Un tableau est une rangée de casiers numérotés

Sans tableau, les ventes de chaque jour sont perdues dès la fin de la boucle. Avec un tableau, on les garde toutes et on les relit à volonté.

```
double ventes[7];
```

ventes[4] vaut 2050 : c'est le vendredi.



Piège classique

Les indices commencent à 0. Le dernier élément d'un tableau de N cases est t[N − 1]. Dans vos boucles, écrivez i < N, jamais i <= N.

Initialiser un tableau

```c
int stock[5] = {120, 45, 0, 300, 18};

int compteurs[10] = {0};      /* tout a 0 */

double prix[] = {15.5, 8.0, 42.9};  /* taille 3 */

#define NB_MOIS 12
double ca[NB_MOIS];
```

Moins de valeurs ?

Les cases restantes valent 0.

Taille omise ?

Le compilateur compte les valeurs.

À retenir

Une constante pour la taille : on la change en un seul endroit.

Parcourir : les jours au-dessus de la moyenne

moyenne.c

```c
for (i = 0; i < N; i++)
{
    printf("Vente du jour %d : ", i + 1);
    scanf("%lf", &ventes[i]);
    total += ventes[i];
}
moyenne = total / N;

for (i = 0; i < N; i++)
    if (ventes[i] > moyenne)
        printf("Jour %d : %.2f\n", i + 1, ventes[i]);
```

Deux passages

1. saisie et total
2. relecture des ventes gardées

Pourquoi i + 1 ?

L'utilisateur préfère lire « jour 1 » plutôt que « jour 0 ».

Trois traitements à connaître par cœur

Maximum

```
int iMax = 0;
for (i = 1; i < N; i++)
  if (t[i] > t[iMax])
    iMax = i;
```

Comptage

```
int nb = 0;
for (i = 0; i < N; i++)
  if (stock[i] == 0)
    nb++;
```

Recherche

```
int pos = -1;
for (i = 0; i < N; i++)
  if (codes[i] == x)
  { pos = i; break; }
```

On garde l'indice du maximum, pas sa valeur : on connaît ainsi le montant et le jour. Pour la recherche, -1 signifie « pas trouvé », car aucun indice valide ne vaut -1.

À retenir

Somme, maximum, comptage, recherche : la même boucle, une condition différente.

Trier par sélection

- 1. Chercher le plus petit élément, l'échanger avec le premier
- 2. Chercher le plus petit des restants, l'échanger avec le deuxième
- 3. Continuer jusqu'à l'avant-dernier

Avant	4200	1500	9800	300
Après	300	1500	4200	9800

tri.c

```
for (i = 0; i < N - 1; i++)
{
    iMin = i;
    for (j = i + 1; j < N; j++)
        if (m[j] < m[iMin])
            iMin = j;
    temp = m[i];
    m[i] = m[iMin];
    m[iMin] = temp;
}
```

Décroissant ? Remplacez < par >.

Tableaux à deux dimensions : une grille

Ventes (kDH)	T1	T2	T3	T4
Errachidia	120	135	150	180
Erfoud	90	85	110	140
Rissani	60	75	70	95

```
double ventes[3][4];
ventes[i][j] : ligne i, colonne j
```

magasins.c

```c
for (i = 0; i < NB_MAG; i++)
{
    totalMag = 0;
    for (j = 0; j < NB_TRIM; j++)
        totalMag += ventes[i][j];
    printf("Magasin %d : %.0f\n",
           i + 1, totalMag);
}
```

Piège classique

Remettre totalMag à 0 au début de chaque ligne, sinon on additionne les magasins entre eux. Résultats : 585, 425 et 300 kDH.

Exercice : les notes d'une classe

Saisir les notes de N étudiants (N ≤ 50), afficher la moyenne, le nombre d'admis et le pourcentage de réussite.

classe.c

```c
for (i = 0; i < n; i++)
{
    printf("Note %d : ", i + 1);
    scanf("%lf", &notes[i]);
    somme += notes[i];
    if (notes[i] >= 10)
        admis++;
}
printf("Moyenne : %.2f\n", somme / n);
printf("Reussite : %.1f %%\n", 100.0 * admis / n);
```

Piège classique

Écrire 100 * admis / n : la division est entière et le pourcentage est tronqué. 100.0 force le calcul en réel.

Exercices 4.1 à 4.5 corrigés dans le polycopié.

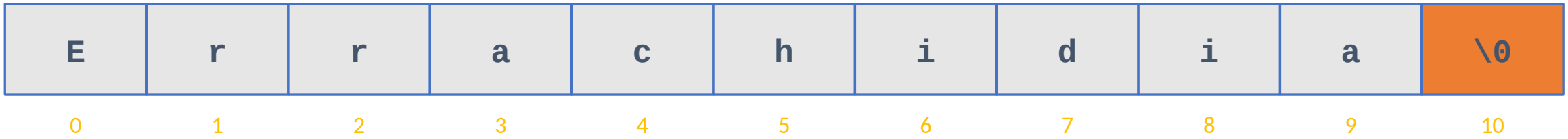
5

Chaînes de caractères

- Une chaîne = un tableau de char + '\0'
- Lire avec scanf et fgets
- Les fonctions de string.h et ctype.h
- Tableaux de chaînes

En mémoire, une chaîne se termine par '\0'

```
char ville[20] = "Errachidia";
```



« Errachidia » : 10 lettres, 11 cases. Il faut toujours N + 1 cases pour un texte de N lettres.
Le caractère nul '\0' marque la fin : c'est grâce à lui que printf sait où s'arrêter.

> **Piège classique**
> 'A' est un caractère (1 case). "A" est une chaîne (2 cases : 'A' puis '\0'). Apostrophes simples pour un caractère, guillemets doubles pour une chaîne.

Lire une chaîne : scanf ou fgets ?

scanf

```
char nom[30];
scanf("%29s", nom);
```

- Pas de & : nom est déjà une adresse
- S'arrête au premier espace : « Societe Atlas » → « Societe »

fgets

```
char adresse[80];
fgets(adresse, sizeof(adresse), stdin);
adresse[strcspn(adresse, "\n")] = '\0';
```

- Lit toute la ligne, espaces compris
- Ne déborde jamais ; la 3e ligne retire le '\n' final

Piège classique

N'utilisez jamais gets : il ne vérifie pas la taille et a été retiré du langage.

La boîte à outils string.h

Fonction	Rôle	Exemple
`strlen(s)`	Longueur, sans '\0'	`strlen("Atlas")  /* 5 */`
`strcpy(d, s)`	Copie s dans d	`strcpy(nom, "Atlas");`
`strcat(d, s)`	Ajoute s à la fin de d	`strcat(nom, " SARL");`
`strcmp(a, b)`	0 si identiques	`strcmp(code, "ADMIN") == 0`
`strchr(s, c)`	Cherche un caractère	`strchr(email, '@')`
`strstr(s, t)`	Cherche un texte	`strstr(libelle, "PROMO")`

Piège classique

strcpy et strcat ne vérifient pas la place disponible : prévoyez des tableaux assez grands.

Comparer deux chaînes : jamais avec ==

```c
/* FAUX : compare deux adresses */
if (nom == "Atlas") ...

/* JUSTE */
if (strcmp(motDePasse, "Caisse@Ziz") == 0)
    printf("Acces autorise.\n");
else
    printf("Mot de passe incorrect.\n");
```

strcmp(a, b)	Signifie
0	a et b identiques
< 0	a avant b (alphabet)
> 0	a après b

À retenir

Copier : strcpy. Comparer : strcmp. Jamais = ni == entre deux chaînes.

ctype.h : tester et transformer un caractère

Fonction	Rôle
`isdigit(c)`	vrai si c est un chiffre
`isalpha(c)`	vrai si c est une lettre
`isupper(c)`	vrai si c est une majuscule
`toupper(c)`	c converti en majuscule
`tolower(c)`	c converti en minuscule

majuscules.c

```c
char nom[] = "El Amrani";
int i;

for (i = 0; nom[i] != '\0'; i++)
    nom[i] = toupper(nom[i]);

printf("%s\n", nom);   /* EL AMRANI */
```

On parcourt une chaîne jusqu'au '\0', sans avoir besoin de connaître sa longueur.

Exercice : contrôler un code client

Un code valide comporte 2 majuscules suivies de 4 chiffres, par exemple ER2045.

code_client.c

```
int valide = 1;
if (strlen(code) != 6)
    valide = 0;
else
{
    for (i = 0; i < 2; i++)
        if (!isupper(code[i])) valide = 0;
    for (i = 2; i < 6; i++)
        if (!isdigit(code[i])) valide = 0;
}
```

Le drapeau

valide vaut 1 au départ et passe à 0 dès qu'une règle n'est pas respectée.

Code saisi	Résultat
ER2045	valide
er2045	invalide
ER20A5	invalide

6

Pointeurs

- Adresse et valeur d'une variable
- Les opérateurs & et *
- Pointeurs et tableaux
- Éviter les plantages : NULL

Chaque variable a une adresse

Dans l'entrepôt, chaque casier a aussi un numéro d'emplacement, « allée 3, étagère 12 ». En mémoire, ce numéro s'appelle l'adresse.

Nom	Adresse (exemple)	Valeur
`stock`	6422300	150
`prix`	6422304	12.5

```c
int stock = 150;
printf("%d\n", stock);           /* 150 */
printf("%p\n", (void *)&stock);  /* 0x7ffd... */
```

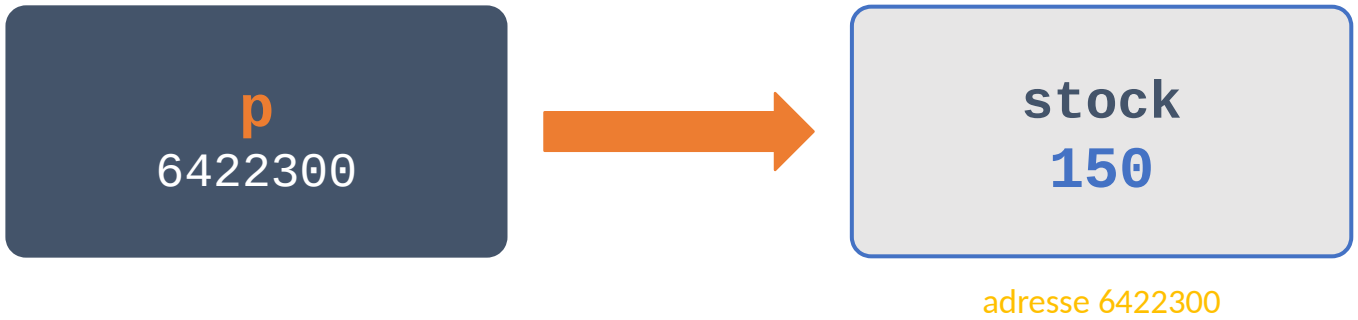
L'opérateur &

&stock donne l'adresse de stock.

C'est le & de scanf : on lui donne l'adresse du casier où déposer la valeur.

Un pointeur contient une adresse

```c
int stock = 150;
int *p = &stock;   /* p pointe sur stock */
```



Écriture	Sens
`int *p;`	p est un pointeur vers un int
`*p = 10;`	la case pointée par p reçoit 10

À retenir

Un pointeur est un post-it sur lequel on a noté l'emplacement d'un casier.

L'étoile a deux rôles : déclarer un pointeur, ou accéder à la valeur pointée.

*p : accéder à la valeur pointée

pointeur.c

```c
int stock = 150;
int *p = &stock;

printf("%d\n", *p);    /* 150 */

*p = *p - 30;          /* modifie stock */
printf("%d\n", stock); /* 120 */
```

Se lit ainsi

« Va à l'adresse notée dans p, prends la valeur, retire 30 et remets le résultat au même endroit. »

À retenir

Si p = &x, alors *p et x désignent la même case. & et * sont deux opérations inverses.

NULL : un pointeur qui ne pointe sur rien

```c
int *p = NULL;

if (p != NULL)
    printf("%d\n", *p);
else
    printf("Ne pointe sur rien.\n");
```

erreur.c

```c
int *p;
*p = 100;       /* PLANTAGE */
```

correction.c

```c
int valeur;
int *p = &valeur;
*p = 100;       /* correct */
```

Piège classique

Utiliser un pointeur qui n'a jamais reçu d'adresse valide : il écrit n'importe où en mémoire (Segmentation fault). Initialisez toujours un pointeur : une adresse ou NULL.

Pointeurs et tableaux : t[i] équivaut à *(t + i)

Expression	Désigne	Valeur
`*p ou prix[0]`	1re case	12.5
`*(p+1) ou prix[1]`	2e case	8.0
`*(p+2) ou prix[2]`	3e case	45.0
`*(p+3) ou prix[3]`	4e case	3.2

```c
double prix[4] = {12.5, 8.0, 45.0, 3.2};
double *p, total = 0;

for (p = prix; p < prix + 4; p++)
    total += *p;     /* 68.70 */
```

À retenir

Le nom d'un tableau est l'adresse de sa première case. p + 1 avance d'un élément, pas d'un octet : le compilateur tient compte du type.

C'est pour cela qu'il n'y a pas de & devant un tableau dans scanf("%s", nom).

Exercice : suivre les cases

```c
int a = 10, b = 20;
int *p = &a;     /* ligne 1 */
*p = 15;         /* ligne 2 */
p = &b;          /* ligne 3 */
*p = *p + a;     /* ligne 4 */
```

Après	a	b	p pointe sur	*p
ligne 1	10	20	a	10
ligne 2	15	20	a	15
ligne 3	15	20	b	20
ligne 4	15	35	b	35

À retenir

Face à un exercice sur les pointeurs, dessinez les cases sur papier et suivez-les ligne par ligne.

À la ligne 4, *p désigne b et a vaut 15 : b reçoit 20 + 15 = 35.

7

Fonctions

- Définir et appeler une fonction
- Prototype et portée des variables
- Passage par valeur et par adresse
- Tableaux en paramètre, récursivité

Pourquoi découper en fonctions ?

Comme une entreprise organisée en services : chacun a son rôle, et le directeur (main) coordonne l'ensemble.

1

Lisibilité

main se lit comme un sommaire, chaque fonction comme un chapitre

2

Réutilisation

une fonction calculerTVA écrite une fois sert partout

3

Fiabilité

on teste chaque fonction séparément et on sait où chercher

Vous en utilisez déjà : printf, scanf, strlen, toupper. Écrivons maintenant les nôtres.

Définir une fonction : le montant TTC

ttc.c

```c
double montantTTC(double ht, double taux)
{
    return ht * (1 + taux / 100);
}

int main(void)
{
    printf("%.2f\n", montantTTC(250, 20));  /* 300.00 */
    printf("%.2f\n", montantTTC(250, 10));  /* 275.00 */
    return 0;
}
```

- **double :** le type du résultat renvoyé
- **ht, taux :** les paramètres reçus
- **return :** renvoie le résultat et termine

À retenir

À l'appel, les arguments sont copiés dans les paramètres, dans l'ordre.

void et prototypes

prototypes.c

```c
/* Prototypes : le sommaire */
double remise(double montant);
void afficherNet(double montant);

int main(void)
{
    afficherNet(6500);   /* 5850.00 */
    return 0;
}

void afficherNet(double montant)
{
    printf("%.2f\n", montant - remise(montant));
}
```

void

La fonction ne renvoie rien : elle agit, par exemple en affichant.

Prototype

Annonce la fonction en haut du fichier ; on l'écrit après main.

Passage par valeur ou par adresse ?

Par valeur : une photocopie

```
void retirer(int stock, int qte)
{
    stock = stock - qte;
}

retirer(stock, 30);
/* stock vaut toujours 100 */
```

Par adresse : l'original

```
void retirer(int *stock, int qte)
{
    *stock = *stock - qte;
}

retirer(&stock, 30);
/* stock vaut 70 */
```

À retenir

Pour qu'une fonction lise une variable, passez sa valeur. Pour qu'elle la modifie, passez son adresse : &x à l'appel, int *p dans les paramètres, *p dans le corps.

Plusieurs résultats grâce aux adresses

facture.c

```c
void calculerFacture(double ht, double *tva, double *ttc)
{
    *tva = ht * 0.20;
    *ttc = ht + *tva;
}

double tva, ttc;
calculerFacture(1500, &tva, &ttc);
printf("TVA %.2f, TTC %.2f\n", tva, ttc);
```

300 | 1 800

TVA et TTC en DH, obtenus en un seul appel

return

ne renvoie qu'une valeur

La fonction d'échange echanger(&x, &y) repose sur le même principe : impossible à écrire sans pointeurs.

Passer un tableau à une fonction

tableaux.c

```c
double somme(const double t[], int n)
{
    double s = 0;
    int i;
    for (i = 0; i < n; i++)
        s += t[i];
    return s;
}

void hausse(double t[], int n, double taux)
{
    int i;
    for (i = 0; i < n; i++)
        t[i] = t[i] * (1 + taux / 100);
}
```

- **Adresse :** le tableau d'origine est modifié directement
- **Taille :** à transmettre en second paramètre
- **const :** promet que la fonction ne modifie pas le tableau

Exemple

Total 750 avant une hausse de 10 %, 825 après.

La récursivité : une fonction qui s'appelle

recursif.c

```c
double valeurAcquise(double c, double taux, int n)
{
    if (n == 0)              /* cas d'arret */
        return c;
    return valeurAcquise(c, taux, n - 1)
           * (1 + taux / 100);
}
```

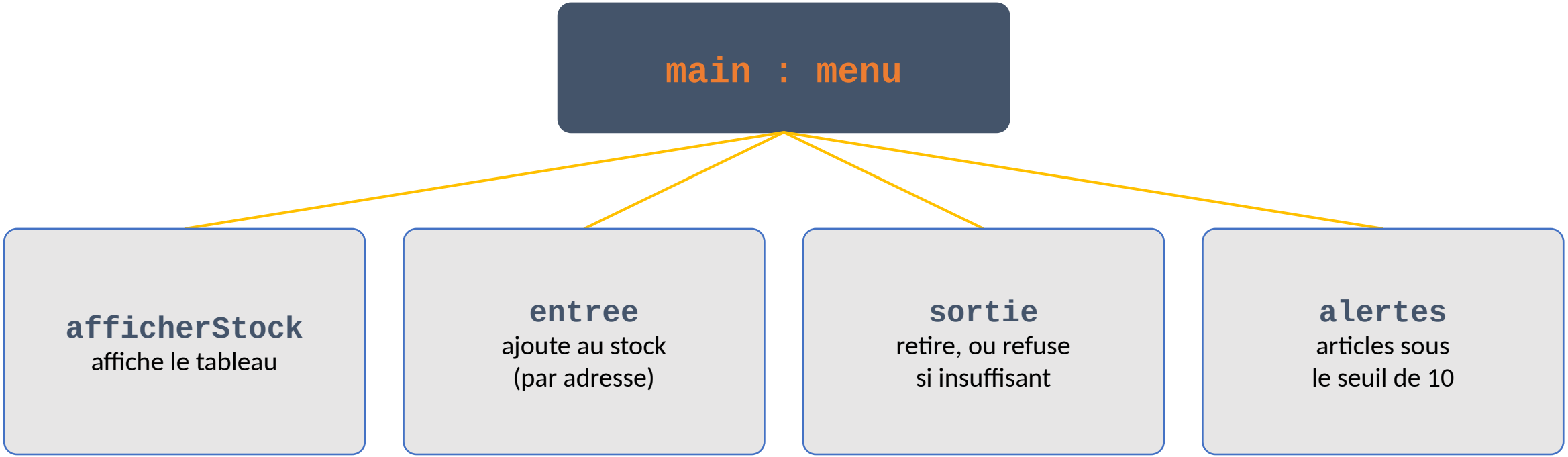
11 576,25 DH

10 000 DH à 5 % pendant 3 ans

La valeur après n années est la valeur après n − 1 années, multipliée par (1 + taux).

> **Piège classique**
>
> Oublier le cas d'arrêt : la fonction s'appelle à l'infini et le programme plante.

Synthèse : un gestionnaire de stock en fonctions



À retenir

Ce programme réunit tout le module : variables, conditions, boucles, tableaux, chaînes, pointeurs et fonctions. Code complet : exercice 7.7 du polycopié.

L'essentiel de chaque chapitre

1. main, accolades, ; à chaque instruction
2. nom, type, valeur ; scanf avec & ; attention à 7 / 2
3. if, switch pour choisir ; for, while pour répéter
4. indices de 0 à N − 1 ; boucle i < N
5. '\0' final ; fgets, strcmp, strcpy
6. &x donne l'adresse, *p la valeur ; t[i] = *(t + i)
7. passer l'adresse pour modifier ; cas d'arrêt

Annexes du polycopié : formats, priorités, dix erreurs fréquentes, sujet d'examen corrigé